

树链剖分

梁宝烨

佛山市南海区石门中学

2025 年 7 月 14 日

目录

1 概念和性质

2 树剖的实现

3 应用

4 练习题

树链剖分，顾名思义，就是把一棵树剖成若干条直上直下的链，以方便解决树上路径的问题。

树链剖分，顾名思义，就是把一棵树剖成若干条直上直下的链，以方便解决树上路径的问题。

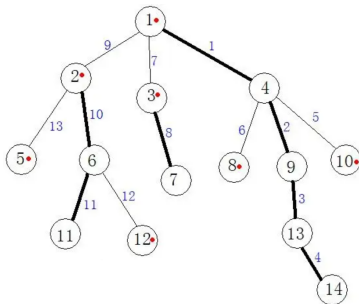
为什么这么说？dfs 序可以使得直上直下的链化为一段连续的区间，于是可以使用序列上的数据结构去维护信息。

一些概念：

- 重儿子：子树大小最大的儿子
- 轻儿子：非重儿子的儿子
- 重边：连向重儿子的边
- 轻边：连向轻儿子的边
- 重链：由重边组成的链
- 轻链：由轻边组成的链

一些概念:

- 重儿子: 子树大小最大的儿子
- 轻儿子: 非重儿子的儿子
- 重边: 连向重儿子的边
- 轻边: 连向轻儿子的边
- 重链: 由重边组成的链
- 轻链: 由轻边组成的链



性质

- 叶节点没有重儿子，非叶节点有且只有一个重儿子。

性质

- 叶节点没有重儿子，非叶节点有且只有一个重儿子。
- 从一个点向上走到根，最多经过 $O(\log n)$ 条轻边。

性质

- 叶节点没有重儿子，非叶节点有且只有一个重儿子。
- 从一个点向上走到根，最多经过 $O(\log n)$ 条轻边。
这个性质非常重要，这意味着，在任意一个点上，如果能快速定位到它所在重链的顶端，那么它很快就能跳到根上。

性质

- 叶节点没有重儿子，非叶节点有且只有一个重儿子。
- 从一个点向上走到根，最多经过 $O(\log n)$ 条轻边。
这个性质非常重要，这意味着，在任意一个点上，如果能快速定位到它所在重链的顶端，那么它很快就能跳到根上。
- 存在一个 dfs 的顺序使得同一条重链的 dfs 序是连续的。

根据定义，我们需要通过第一遍的 dfs，把每个节点的重儿子找出来。

根据定义，我们需要通过第一遍的 dfs，把每个节点的重儿子找出来。

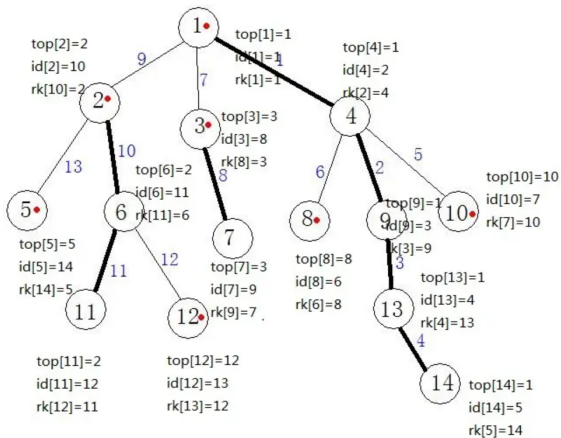
```
1 void dfs1(int u){ // 在dfs的过程中顺便记录父亲和深度
2     dep[u]=dep[f[u]]+1;
3     size[u]=1; // 初始化子树大小
4     for(int i=head[u];i;i=edge[i].next)
5         if(edge[i].target!=f[u]){
6             int v=edge[i].target;
7             f[v]=u;
8             dfs1(v);
9             size[u]+=size[v];
10            if(size[v]>size[son[u]]) son[u]=v; // 更新重儿子
11        }
12 }
```

第二遍 dfs，我们需要连接重链（即记录 top），以及保证一条重链上各个节点 dfs 序连续。

第二遍 dfs, 我们需要连接重链 (即记录 top), 以及保证一条重链上各个节点 dfs 序连续。

```
1 void dfs2(int u,int t){ // 当前节点是u, u所在重链的顶端是t
2     top[u]=t;
3     id[u]=++cnt, rk[cnt]=u; // 更新dfs序
4     if(!son[u]) return;
5     dfs2(son[u],t); // 为保证重链dfs序连续的性质, 需要优先遍
        历重儿子
6     for(int i=head[u];i;i=edge[i].next){
7         int v=edge[i].target;
8         if(v!=son[u]&&v!=f[u]) dfs2(v,v); // 轻儿子自己新开
            了一条重链
9     }
10 }
```

dfs 跑完大概是这样。



求 LCA

说了这么多，你这树剖有啥用？

求 LCA

说了这么多，你这树剖有啥用？
还记得之前说的性质吗，可以利用它来求 LCA。

求 LCA

说了这么多，你这树剖有啥用？

还记得之前说的性质吗，可以利用它来求 LCA。

具体而言，若要求 $LCA(x, y)$ ，若 x, y 在同一条重链上，那他们一定构成祖先关系。

求 LCA

说了这么多，你这树剖有啥用？

还记得之前说的性质吗，可以利用它来求 LCA。

具体而言，若要求 $LCA(x, y)$ ，若 x, y 在同一条重链上，那他们一定构成祖先关系。

否则，类似倍增求 LCA 一样，我们让深度更大的往上跳，假设是 x ，则令 $x = fa[top[x]]$ 。

求 LCA

说了这么多，你这树剖有啥用？

还记得之前说的性质吗，可以利用它来求 LCA。

具体而言，若要求 $LCA(x, y)$ ，若 x, y 在同一条重链上，那他们一定构成祖先关系。

否则，类似倍增求 LCA 一样，我们让深度更大的往上跳，假设是 x ，则令 $x = fa[top[x]]$ 。

因此时间复杂度是 $O(\log n)$ 。

求 LCA

说了这么多，你这树剖有啥用？

还记得之前说的性质吗，可以利用它来求 LCA。

具体而言，若要求 $LCA(x, y)$ ，若 x, y 在同一条重链上，那他们一定构成祖先关系。

否则，类似倍增求 LCA 一样，我们让深度更大的往上跳，假设是 x ，则令 $x = fa[top[x]]$ 。

因此时间复杂度是 $O(\log n)$ 。

```
1 int lca(int a, int b){  
2     while(top[a] != top[b]) depth[top[a]] < depth[top[b]] ? b = f[  
        top[b]] : a = f[top[a]];  
3     return depth[a] < depth[b] ? a : b;  
4 }
```

树上路径问题

由于树剖剖的是直上直下的链，对于树上的一条路径，它可以被表示成 $O(\log n)$ 条重链的并。

树上路径问题

由于树剖剖的是直上直下的链，对于树上的一条路径，它可以被表示成 $O(\log n)$ 条重链的并。

证明同上，假设路径的两端是 x, y ，那么它们通过跳重链跳到 $lca(x, y)$ 只要 $O(\log n)$ 次。

树上路径问题

由于树剖剖的是直上直下的链，对于树上的一条路径，它可以被表示成 $O(\log n)$ 条重链的并。

证明同上，假设路径的两端是 x, y ，那么它们通过跳重链跳到 $lca(x, y)$ 只要 $O(\log n)$ 次。

而跳的过程就是用重链对路径进行拆解。

树上路径问题

伪代码：

树上路径问题

伪代码：

```
1 void range(int a, int b, ...) {  
2     while(top[a] != top[b]){  
3         if(dep[top[a]] > dep[top[b]]) swap(a, b);  
4         .... // 可以对路径进行操作  
5         b = f[top[b]];  
6     }  
7     if(id[a] > id[b]) swap(a, b);  
8     .... // 可以对路径进行操作  
9 }
```

树上路径问题

还记得上面提到的第三条性质吗？

树上路径问题

还记得上面提到的第三条性质吗？

因为重链上的点 dfs 序连续，相当于我们把路径划分成 $O(\log n)$ 个序列上的区间了！

树上路径问题

还记得上面提到的第三条性质吗？

因为重链上的点 dfs 序连续，相当于我们把路径划分成 $O(\log n)$ 个序列上的区间了！

那么对于一些树上路径的操作，就可以转化成序列上来看（即可以用一些维护序列的数据结构（如线段树）来维护树上路径的信息）

树上路径问题

还记得上面提到的第三条性质吗？

因为重链上的点 dfs 序连续，相当于我们把路径划分成 $O(\log n)$ 个序列上的区间了！

那么对于一些树上路径的操作，就可以转化成序列上来看（即可以用一些维护序列的数据结构（如线段树）来维护树上路径的信息）

此外，由于 dfs 序本身的性质，某个点及其子树构成的 dfs 序也是连续的一段区间，因此也是可以同时维护子树内的信息。

- 模板题：P3384 【模板】重链剖分/树链剖分，P2590 [ZJOI2008] 树的统计
- 求 LCA：P3379 【模板】最近公共祖先 (LCA)

P2486 [SDOI2011] 染色

P2486 [SDOI2011] 染色

这道题大概是想提醒大家，有时候考察点不在树剖上，而是在其使用的数据结构上。

P2486 [SDOI2011] 染色

这道题大概是想提醒大家，有时候考察点不在树剖上，而是在其使用的数据结构上。

用线段树的时候，应该维护区间最左颜色和区间最右颜色，毕竟统计颜色段数量时不能简单地区间加法。

P3979 遥远的国度

P3979 遥远的国度

首先，根无论在哪里，树上任意两点的路径都是固定的。

P3979 遥远的国度

首先，根无论在哪里，树上任意两点的路径都是固定的。
因此我们只需考虑子树修改的操作即可。

P3979 遥远的国度

首先，根无论在哪里，树上任意两点的路径都是固定的。
因此我们只需考虑子树修改的操作即可。
我们以 1 为根进行树剖，假设现在的根是 $root$ ，要修改子树 u 。
我们基于以 1 为根分类讨论一下：

- $root = u$ ：改全部点。
- $root$ 不在 u 的子树内：那和以 1 为根没区别。
- $root$ 在 u 的子树内，那么现在 u 新的子树就是除去 u 往 $root$ 方向后，剩下的所有点。

显然这些都可以用 dfs 序上的连续区间来表示子树。

P4211 [LNOI2014] LCA

P4211 [LNOI2014] LCA

bouns: 树上差分。